

III. Les attributs HTML

LES ATTRIBUTS EN UNE PHRASE

Des paires `nom="valeur"` qui enrichissent une balise en précisant son comportement, son apparence ou son rôle — par exemple `<a href="...">` ou `<img alt="...">`.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Anatomie d'un attribut**
- **B. Attributs globaux**
 - Les attributs `data-*`
- **C. Attributs des liens et images**
 - 1. Liens hypertextes (`a`)
 - 2. Images (`img`)
- **D. Attributs de formulaire**
 - 1. Champs de saisie (`input`)
 - 2. `textarea` et `select`
- **E. Attributs multimédia et tableaux**
 - 1. `audio` et `video`
 - 2. Attributs de table
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Identifier** les attributs globaux (utilisables sur toute balise) vs les attributs spécifiques.
- **Appliquer** les attributs `href`, `src`, `alt`, `type`, `required`, `pattern` dans leurs contextes respectifs.
- **Distinguer** `id` (unique) et `class` (multiple) et leurs usages respectifs.
- **Écrire** une balise `<img>` ou `<a>` accessible et performante (`alt`, `loading`, `rel`).

2. En pratique — situation professionnelle

Un audit Lighthouse remonte sur votre site : *Images don't have `loading="lazy"`*, *Links with `target=_blank` are a security risk*. Les deux corrections tiennent en un attribut chacune — à condition de savoir lesquels utiliser et pourquoi.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
<code>class</code> pour le style et la réutilisation (90 % des cas)	<code>id</code> pour styliser : trop spécifique, difficile à surcharger
<code>data-*</code> pour passer des infos du HTML à JS sans hack	<code>style=""</code> inline en production (sauf JS dynamique)
<code>rel="noopener"</code> systématique avec <code>target="_blank"</code> externe	Omettre <code>alt</code> sur <code></code> : audit accessibilité KO

A. Anatomie d'un attribut

Un attribut HTML a toujours la forme `nom="valeur"` et se place **dans la balise ouvrante** :

```
<a href="https://diginamic.fr" target="_blank" rel="noopener">Site Diginamic</a>
```

Règles de syntaxe :

- Le nom de l'attribut est en **minuscules** (`href` , pas `HREF`).
- La valeur est entourée de **guillemets doubles** `"..."` (préféré en HTML5).
- Plusieurs attributs sont séparés par un **espace**.
- Certains attributs sont **booléens** et n'ont pas de valeur (`<input required>` , `<video autoplay>` , `<button disabled>`).

B. Attributs globaux

Utilisables sur **n'importe quelle balise** HTML :

Attribut	Rôle	Exemple
<code>class</code>	Une ou plusieurs classes CSS (séparées par espaces)	<code><p class="intro highlight"></code>
<code>id</code>	Identifiant unique dans la page	<code><main id="contenu-principal"></code>
<code>lang</code>	Langue du contenu	<code><blockquote lang="en"></code>
<code>title</code>	Info-bulle au survol	<code><abbr title="World Wide Web">WWW</abbr></code>
<code>hidden</code>	Cache l'élément	<code><div hidden></code> (attribut booléen)
<code>style</code>	CSS inline (à éviter en production)	<code><p style="color: red;"></code>
<code>data-*</code>	Données personnalisées pour JavaScript	<code><button data-user-id="42"></code>

♦ `id` vs `class`

- `id` = **unique** dans toute la page. Utilisé pour cibler un élément précis (ancrage, JavaScript, label).
- `class` = **réutilisable** autant qu'on veut. Utilisé pour appliquer des styles ou regrouper des éléments.

Règle : si vous hésitez, prenez `class`.

Les attributs `data-*`

```
<button data-user-id="42" data-role="admin">Supprimer</button>
```

Accessible en JavaScript via `dataset` :

```
const btn = document.querySelector('button');
console.log(btn.dataset.userId); // "42"
console.log(btn.dataset.role); // "admin"
```

C. Attributs des liens et images

1. Liens hypertextes (`a`)

```
<a href="https://example.com"
  target="_blank"
  rel="noopener noreferrer"
  hreflang="fr"
  download>
  Télécharger le rapport
</a>
```

Attribut	Rôle
<code>href</code>	URL de destination (obligatoire pour un vrai lien)
<code>target="_blank"</code>	Ouvre dans un nouvel onglet
<code>rel="noopener noreferrer"</code>	OBLIGATOIRE avec <code>target="_blank"</code> pour la sécurité
<code>download</code>	Force le téléchargement au lieu de l'affichage
<code>hreflang</code>	Langue de la page liée

⚠ Sécurité des `target="_blank"`

Sans `rel="noopener"`, la page ouverte peut accéder à `window.opener` et modifier la page d'origine (attaque *tabnabbing*). **Toujours** ajouter `rel="noopener noreferrer"` avec un `target="_blank"` vers un site externe.

2. Images (`img`)

```

```

Attribut	Rôle
<code>src</code>	Source de l'image (obligatoire)
<code>alt</code>	Texte alternatif (obligatoire, même vide pour les décoratives)
<code>width</code> / <code>height</code>	Dimensions en pixels — préférable pour éviter le <i>layout shift</i>
<code>loading="lazy"</code>	Charge l'image seulement quand elle approche du viewport — bon pour la perf

♦ `loading="lazy"` par défaut

Sauf pour les images *above the fold* (visibles à l'ouverture sans scroll), toujours ajouter `loading="lazy"`. Gain de performance immédiat.

D. Attributs de formulaire

1. Champs de saisie (`input`)

```
<input type="email"
      name="email"
      id="email"
      required
      placeholder="vous@example.com"
      minlength="5"
      maxlength="100"
      autocomplete="email">
```

Attribut	Rôle
<code>type</code>	Type de champ (text, email, password, number, date, tel, url, file, range, color...)
<code>name</code>	Nom du champ (utilisé lors de la soumission)
<code>id</code>	Identifiant unique (pour lier avec <code><label></code>)
<code>required</code>	Champ obligatoire (booléen)
<code>placeholder</code>	Texte d'exemple (pas un label !)
<code>pattern</code>	Validation par expression régulière
<code>min</code> / <code>max</code> / <code>step</code>	Pour les types numériques
<code>autocomplete</code>	Permet/bloque le remplissage automatique du navigateur

Types d'input HTML5

HTML5 a introduit beaucoup de types qui améliorent l'UX mobile (clavier adapté) et la validation native : `email` , `tel` , `url` , `date` , `number` , `range` , `color` , `search` . Les utiliser systématiquement au lieu de `type="text"` générique.

2. textarea et select

```
<textarea name="message"
  rows="4"
  cols="50"
  spellcheck="true"
  placeholder="Votre message..."></textarea>

<select name="pays" multiple size="3">
  <option value="fr" selected>France</option>
  <option value="be">Belgique</option>
  <option value="ch" disabled>Suisse (indisponible)</option>
</select>
```


E. Attributs multimédia et tableaux

1. audio et video

```
<video src="video.mp4"
      width="640" height="360"
      poster="aperçu.jpg"
      controls
      preload="metadata">
  Votre navigateur ne supporte pas la vidéo.
</video>
```

Attribut	Rôle
<code>controls</code>	Affiche les contrôles natifs (play, pause, volume)
<code>autoplay</code>	Lecture automatique (souvent bloqué par les navigateurs sans <code>muted</code>)
<code>loop</code>	Relance en boucle
<code>muted</code>	Son coupé (requis pour autoplay moderne)
<code>poster</code>	Image d'aperçu avant lecture (vidéo seulement)
<code>preload="metadata"</code>	Charge juste les métadonnées, pas la vidéo entière

2. Attributs de table

```
<table>
  <thead>
    <tr>
      <th colspan="2">En-tête fusionné sur 2 colonnes</th>
      <th>Autre colonne</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td rowspan="3">Fusionné sur 3 lignes</td>
      <td>A</td>
      <td>1</td>
    </tr>
  </tbody>
</table>
```

Attribut	Rôle
<code>colspan</code>	Fusion sur plusieurs colonnes
<code>rowspan</code>	Fusion sur plusieurs lignes
<code>headers</code>	Lie une cellule à ses en-têtes (accessibilité)

Conclusion

1. À retenir

- Syntaxe : `<balise nom="valeur">`, toujours en **minuscules**, guillemets doubles.
- **Attributs globaux** (class, id, lang, title, data-*) : utilisables partout.
- `alt` est **obligatoire** sur toute `<img>` (vide pour les décoratives).
- `target="_blank"` → **toujours ajouter** `rel="noopener noreferrer"`.
- `loading="lazy"` par défaut sur les images non visibles à l'ouverture.
- Utiliser les **types HTML5** (`email`, `tel`, `date`, ...) au lieu de `type="text"` générique.
- Les `data-*` stockent des infos custom, accessibles via `element.dataset.*` en JS.

2. Pour aller plus loin

- **MDN — Attributs HTML (référence complète)**
- **MDN — Attributs globaux**
- **Can I Use — compatibilité navigateurs par attribut**
- **OWASP — `rel="noopener"` security**